

Term Project – Proposal

Hotel Booking Manager

Dozent: Stephan Bökelmann sboekelmann@ep1.rub.de Technische Hochschule Georg Agricola, Bochum

Author: Abdelillah Taleb

Module: Introduction to Database Management Systems · THGA Bochum

Submitted to: Stephan Bökelmann · sboekelmann@ep1.rub.de

Target size: about 6 pages

1 Application domain and motivation

The Hotel Booking Manager is a desktop application designed to simplify the management of hotel reservations and room administration. Instead of storing guest information and bookings in spreadsheets or on paper, the application keeps all data in a PostgreSQL database. This improves data consistency, reduces booking errors and makes daily hotel operations more efficient.

The application is intended for receptionists and hotel managers. Receptionists can register guests, create bookings, assign rooms and update reservations. Managers can monitor room occupancy and access booking statistics generated from the stored data.

The following scenarios describe the intended use of the application:

- Guest check-in.

A receptionist searches for an existing guest or registers a new one, selects an available room and creates a booking for the requested dates. The booking is stored immediately and becomes visible in the booking overview.

- Daily hotel administration.

The hotel manager reviews current bookings, checks room availability and views statistical information such as the total number of bookings and the occupancy of the hotel. This information helps to organize daily operations and support business decisions.

2 Data model – ER sketch

Figure 1 illustrates the database structure of the Hotel Booking Manager. The system is based on six entities: Guest, Room Category, Room, Booking, Service and Booking_Service. Each guest may have multiple bookings, while every booking belongs to exactly one guest. Every room is assigned to one room category, and each room category may contain several rooms. A booking is linked to one room, ensuring that reservations are assigned correctly. The relationship between Booking and Service is many-to-many because a booking may include several hotel services, and each service may be associated with multiple bookings. This relationship is implemented through the Booking_Service entity, which stores the selected service together with its quantity.

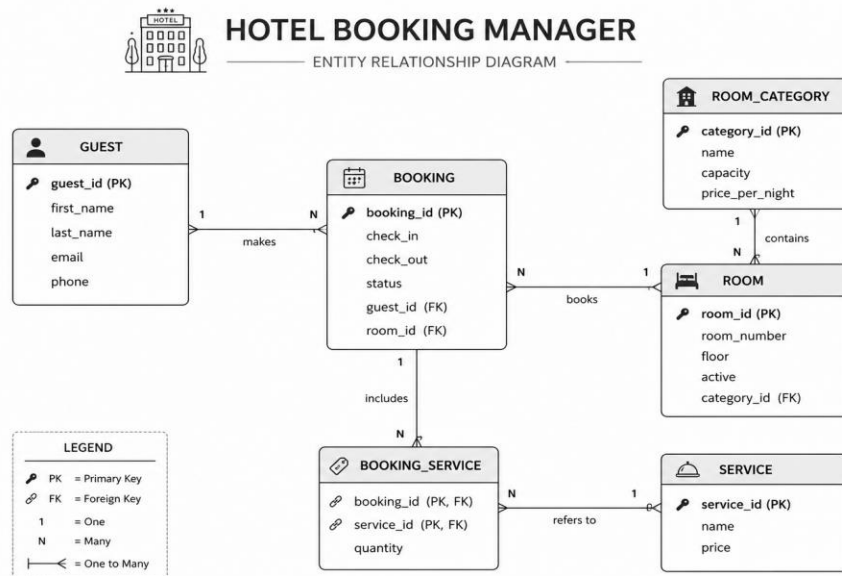


Abbildung 1: Entity Relationship Diagram of the Hotel Booking Manager showing the entity types, primary keys, relationships and cardinalities.

3 From model to schema

The relational database schema is derived from the Entity Relationship Diagram presented in Figure 1. Each entity is represented by an individual database table. Relationships between entities are implemented using foreign keys, while the many-to-many relationship between bookings and services is resolved through the associative table Booking_Service.

- guest(guest_id, first_name, last_name, email, phone)
- room_category(category_id, name, capacity, price_per_night)
- room(room_id, room_number, floor, active, category_id)
- booking(booking_id, check_in, check_out, status, guest_id, room_id)
- service(service_id, name, price)
- booking_service(booking_id, service_id, quantity)

Foreign Keys: • room.category_id references room_category.category_id

- booking.guest_id references guest.guest_id
- booking.room_id references room.room_id
- booking_service.booking_id references booking.booking_id
- booking_service.service_id references service.service_id

Normalisation: The database structure follows the principles of the Third Normal Form (3NF). Each table represents one specific entity and stores only information that belongs to that entity. Data that could otherwise appear multiple times, such as room categories or hotel services, is stored once and linked through foreign keys. This design reduces redundancy, improves consistency and simplifies future maintenance of the database.

4 API design

The application backend will be implemented with FastAPI and communicates with a PostgreSQL database. All requests require a valid API key, which is transmitted in the HTTP header using the X-API-Key field. The API follows REST principles and exchanges data in JSON format.

Method	Path	Purpose
GET	/rooms	Returns a list of all rooms together with their category and availability.
GET	/bookings	Returns all bookings including guest and room information.
GET	/statistics	Returns statistical information such as the total number of bookings and occupied rooms.
POST	/services	Adds a hotel service to an existing booking.
POST	/bookings	Creates a new booking after validating the selected room and booking dates.

5 Frontend prototype (hand sketches)

The desktop application provides a simple interface for hotel employees. After establishing a connection to the database, users can manage reservations through the booking overview. New bookings can be created using a dedicated form, while existing reservations can be inspected and updated in the details view. A separate statistics page displays booking-related information generated from SQL aggregation queries, supporting management decisions and providing an overview of hotel occupancy.

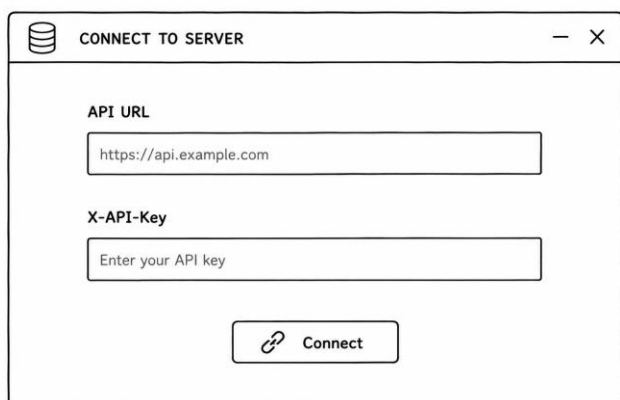


Abbildung 2: Startup connection dialog: API URL + X-API-Key.

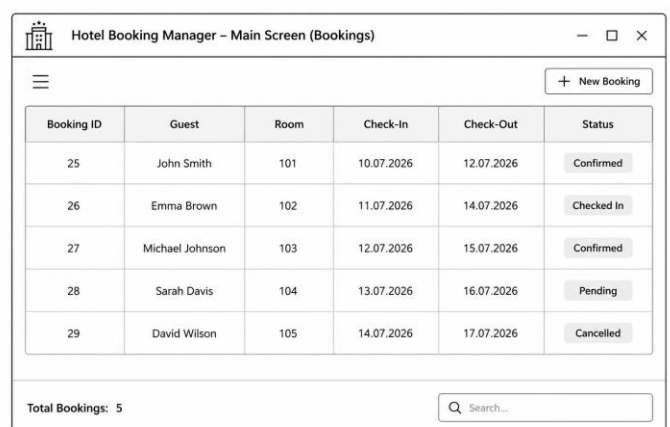


Abbildung 3: Booking overview screen: list of reservations, calls GET /bookings.

New Booking

Guest:

Room:

Check-In:

Check-Out:

Status:

Abbildung 4: Create-booking form: guest, room and dates, calls POST /bookings

Booking Details

[← Back to List](#)

Booking Information

Booking ID : 25
 Guest : John Smith
 Room : 101 (Single Room)
 Room Category : Standard
 Check-In : 10.07.2026
 Check-Out : 12.07.2026
 Status : Confirmed
 Created At : 08.07.2026 14:30

Services

Service	Quantity	Price	Total
Breakfast	2	€ 20.00	€ 40.00
Airport Transfer	1	€ 30.00	€ 30.00
Total			€ 70.00

Abbildung 5: Booking details screen: reservation information and booked services, calls GET /bookings

Figures 2–5 present the planned user interface of the Hotel Booking Manager. Figure 2 shows the startup connection window, where the user enters the API URL and X-API-Key to establish a connection with the backend. Figure 3 presents the main booking overview, displaying existing reservations and allowing users to navigate through the application. Figure 4 illustrates the form used to create a new booking by selecting a guest, room and reservation details. Figure 5 shows the booking details view, where users can inspect reservation information together with the associated hotel services.

6 Operation with Docker Compose

The application is deployed with Docker Compose. The configuration defines two main services: a PostgreSQL database container and a FastAPI backend container. The database service stores all hotel, guest and booking data, while the API service provides the REST endpoints used by the desktop application.

Database credentials, the database name and the X-API-Key are not written directly into the Docker Compose file. They are stored in a separate `.env` file and loaded as environment variables when the containers are started. The `.env` file will not be committed to the public repository; instead, an example configuration file will document the required variables.

On the lecture server, the repository is cloned and configured with the server-specific `.env` file. The backend and database are then built and started using `docker compose up -d --build`. The FastAPI service connects to PostgreSQL through the internal Docker network and exposes the API port required by the desktop frontend.

7 Scope, milestones and risks

Scope. The project consists of six database tables, five REST API endpoints, a PostgreSQL database, a FastAPI backend and a desktop frontend developed with Python/tkinter. The application supports hotel booking management, room administration and booking statistics.

Milestones and schedule.

Week	Milestone	Est. hours
1	Design the database schema, create the ER model and implement the PostgreSQL database with sample data.	8
2–3	Develop the FastAPI backend, implement REST endpoints and API key authentication.	12
4–5	Implement the desktop frontend, connect it to the backend and prepare the Docker deployment.	10
6	Test the application, fix remaining issues and complete the documentation and demonstration video.	10

Risks. Possible risks include unexpected integration issues between the frontend and backend, as well as challenges in validating room availability to prevent overlapping bookings. Additional time may also be required for debugging and testing the complete application.

8 Acceptance criteria and testing

This section defines the conditions under which the project is considered complete and correct. Each acceptance criterion can be verified through a corresponding test case, ensuring that the implemented functionality meets the project requirements. The identified project risk is also covered by a dedicated business rule test.

Acceptance criteria.

- A booking request for a room that is already reserved during the selected period is rejected (API returns 4xx).
- GET /statistics returns the correct booking and room occupancy statistics based on SQL JOIN and aggregation queries.
- POST /bookings requests without a valid X-API-Key are refused (401/403).

Test plan.

- Database: Verify PRIMARY KEY, FOREIGN KEY, NOT NULL and referential integrity constraints for all database tables.
- API: Test all GET and POST endpoints with valid and invalid input using pytest and FastAPI's TestClient against a test database.

- **Business rule:** Verify that overlapping bookings for the same room cannot be created, while bookings for available rooms are accepted successfully.

Effort budget: the whole term project (system, documentation and video) should fit into roughly **40h of work** over at most **2 months**. Keep your scope within that budget.